

Two-Phase PPO Employee Scheduling with GNN Actor-Critic

This notebook solves the employee scheduling problem using **Proximal Policy Optimization (PPO)** with a **GNN (Graph Neural Network) Actor-Critic**. The environment is modeled as a Gym where the agent makes one shift-assignment decision per (employee, day) pair, split across two phases.

Problem Constraints

- **223 workdays** per employee per year
- Max **5 consecutive** workdays
- Max **22 special day** (holiday/weekend) assignments
- Employees can only be assigned to **their own teams**
- No backward shift transitions (e.g., Afternoon → Morning the next day)
- Respect **vacation days**
- Meet **minimum staffing demand** per shift/team each day

Environment (ScheduleEnv)

Two-Phase Design

Each episode runs in two sequential phases:

Phase 1 — Coverage Assignment: The agent processes all (employee, day) pairs in a shuffled day order. A **hard coverage mask** prevents assigning shifts where `daily_coverage >= min_demand` — the agent can only fill actual gaps or choose REST. This phase focuses on meeting minimum staffing demand.

Phase 2 — Gap Filling & Budget Completion: After Phase 1, the environment collects all (employee, day) pairs where the employee is resting and still has budget remaining. **Shortfall days come first** (priority), then non-shortfall days. The agent decides whether to assign a shift or keep resting, targeting: (1) reduce remaining shortfall, (2) bring all employees to 223 working days.

```
def _build_phase2_steps(self):
    """Build list of (emp, day) pairs for Phase 2.
    Shortfall days first (priority), then non-shortfall days (fill to
    223)."""
    shortfall_steps = []
    fill_steps = []

    for day in range(self.num_days):
        has_gap = False
        for s in range(2):
            for t in range(2):
                if self.daily_coverage[day, s, t] < self.min_demand[day, s,
t]:
                    has_gap = True
                    break
```

```
        if has_gap:
            break

    for emp in range(self.num_employees):
        if self.days_worked[emp] >= self.max_days_per_year:
            continue
        row = self._row(emp, day)
        slot = self.state[row, 2:]
        if slot[0] != 1: # not resting
            continue

        if has_gap:
            shortfall_steps.append((emp, day))
        else:
            fill_steps.append((emp, day))

    return shortfall_steps + fill_steps
```

State Representation

The state is a **day-major matrix** of shape `(num_employees × num_days, 2 + 5)`. Day order is shuffled each episode via `np.random.permutation`:

Row	emp	day	REST	M-A	T-A	M-B	T-B
0	0	d0	0	0	0	0	0
1	1	d0	0	0	0	0	0
...							

Action Space

5 discrete actions per step:

Action	Meaning
0	Rest (day off)
1	Morning shift, Team A
2	Afternoon shift, Team A
3	Morning shift, Team B
4	Afternoon shift, Team B

Illegal actions are masked (set to `-inf` before softmax) so the agent can never pick them.

Action Masking

Both phases enforce the same base constraints:

- Vacation days → only REST
- Max 223 days reached → only REST
- Max 5 consecutive days → only REST
- Special day cap (22) reached → only REST
- Shift transitions (no Afternoon → Morning next day)
- Team eligibility (employees only assigned to their teams)

Phase 1 additionally masks any shift-team where `daily_coverage >= min_demand`, preventing over-assignment.

Reward Functions

Phase 1 — Step Reward

Scenario	Reward	Rationale
Work & dual-team employee picks team B	+0.5	Encourages flexible team B usage
End of day, all demand met	+3.0	Bonus for fully covered day
End of day, demand unmet	-2.0 × shortfall	Penalty per missing slot

The coverage mask already prevents over-assignment, so every work action in Phase 1 fills a real gap.

Phase 2 — Step Reward (Tiered)

Scenario	Reward	Rationale
Fill a shortfall gap (cov <= demand)	+3.0	Primary goal — reduce shortfall
Work on non-shortfall day (toward 223)	+1.0	Secondary goal — reach 223 days
Rest on shortfall day	-0.5	Missed opportunity to close gap
Rest on non-shortfall day	-0.1	Mild penalty
Dual-team employee picks team B	+0.5	Same bonus as Phase 1

Final Reward (end of episode)

```
def _calculate_final_reward(self):
    reward = 0.0
    total_shortfall = 0
    for day in range(self.num_days):
        for s in range(2):
            for t in range(2):
                shortfall = self.min_demand[day, s, t] -
self.daily_coverage[day, s, t]
                if shortfall > 0:
                    total_shortfall += shortfall
    reward -= total_shortfall * 10.0           # Primary: penalize unmet demand
```

```

if total_shortfall == 0:
    reward += 500.0 # Bonus for perfect coverage

# Secondary: penalize distance from 223 target
for emp in range(self.num_employees):
    deficit = self.max_days_per_year - self.days_worked[emp]
    if deficit > 0:
        reward -= deficit * 1.0

return reward

```

Weight ratio: 10.0 (shortfall) vs 1.0 (223-deficit) ensures shortfall minimization remains the primary objective.

Network Architecture (GNNActorCritic)

The network has two distinct paths that merge at the actor/critic heads:

1. **GNN path** — processes graph node features through message passing, producing embeddings that capture relational context across all employees and days. Updated **once per day**.
2. **Direct path** — live per-step features (coverage gaps, dynamic features, phase) that bypass the GNN entirely and go straight to the heads. Updated **every step**.

```

GNN path:      emp_node_feats(3) → GNN layers → emp_emb(64) —┐
                day_node_feats(6) → GNN layers → day_emb(64) —┐
                                                           └─┬─┐
concat(135) → Actor → action probs
Direct path:  cov_gap(4) —————┐
Critic → value  dyn_feat(2) —————┐
                phase(1)  —————┐

```

GNN Path (updated once per day)

Graph Structure

A **bipartite heterograph** built once at initialization:

- **12 employee nodes** $\leftrightarrow$ **365 day nodes**
- Every employee connected to every day via `assigned_to` / `staffed_by` edges

Node Features

These are set by `update_graph_features(graph, env)`, which runs **once per day change** during trajectory collection. They represent a snapshot of the environment state at that point.

Employee nodes (3 features):

Feature	Value	Description
days_worked / 223	0.0 → 1.0	How much of the work budget is used
consecutive_streak / 5	0.0 → 1.0	How many working consecutive days
is_dual_team	0.0 or 1.0	Static — whether the employee belongs to both teams

Day nodes (6 features):

Feature	Value range	Description
cov_gap_MA	-N to +3	<code>min_demand</code> - <code>coverage</code> for Morning, Team A
cov_gap_TA	-N to +3	<code>min_demand</code> - <code>coverage</code> for Afternoon, Team A
cov_gap_MB	-N to +3	<code>min_demand</code> - <code>coverage</code> for Morning, Team B
cov_gap_TB	-N to +3	<code>min_demand</code> - <code>coverage</code> for Afternoon, Team B
is_special	0.0 or 1.0	Static — holiday or weekend
day_position	0.0 → 1.0	Static — normalized position in year (day / 365)

Message Passing (2 layers)

Raw node features are first projected to a shared hidden dimension:

```
Employee feats (3) → Linear(3, 32)
Day feats (6)      → Linear(6, 32)
```

Then two rounds of message passing via `HeteroGraphConv` with `SAGEConv(mean)`:

- **Layer 1** (32 → 32): Each employee node receives the mean of all 365 day vectors and updates itself. Each day node receives the mean of all 12 employee vectors and updates itself. ReLU activation.
- **Layer 2** (32 → 64): Same process, output expands to 64 dimensions. ReLU activation.

The result: every employee has a **64-dim embedding** that's informed by all days, and every day has a **64-dim embedding** informed by all employees. These embeddings capture global relational context — e.g., an employee's embedding "knows" not just its own budget usage, but also how much demand remains across all days.

Caching

The GNN forward pass runs **once per day change**, and the resulting embeddings are cached:

```
if day_id != cached_day_id:
    update_graph_features(graph, env)
    cached_emp_emb, cached_day_emb = model.gnn_forward(graph)
    cached_day_id = day_id
```

When processing day 50 with 12 employees, the GNN runs once before employee 0, then employees 1–11 reuse the same cached embeddings. This means the embeddings become slightly stale within a day (e.g., the GNN thinks day 50's Morning-A gap is 3, but after 2 assignments it's actually 1). The direct path compensates for this.

Direct Path (updated every step)

These features are computed **fresh from the environment at every step**, right before each action decision:

```
cov_gap = [
    env.min_demand[day_id, 0, 0] - env.daily_coverage[day_id, 0, 0], # M-A
    env.min_demand[day_id, 1, 0] - env.daily_coverage[day_id, 1, 0], # T-A
    env.min_demand[day_id, 0, 1] - env.daily_coverage[day_id, 0, 1], # M-B
    env.min_demand[day_id, 1, 1] - env.daily_coverage[day_id, 1, 1], # T-B
]
consec = _get_consecutive_days(env, emp_id, day_id)
dyn_feat = [env.days_worked[emp_id], consec]
```

Feature	Dims	Updated	Purpose
Coverage gaps	4	Every step	Exact current gaps for today's shift-teams
days_worked	1	Every step	Employee's current budget usage
consecutive_days	1	Every step	Real consecutive work streak (computed from schedule)
phase	1	At phase transition	0.0 for Phase 1, 1.0 for Phase 2

These bypass the GNN and go directly to the heads. They **correct for the staleness** of the cached GNN embeddings — most importantly the coverage gaps, which can change multiple times within a single day as employees get assigned.

Actor-Critic Heads

The heads receive the concatenation of both paths:

```
cached_emp_emb[emp_id]    (64) — from GNN, updated per day
cached_day_emb[day_id]    (64) — from GNN, updated per day
cov_gap                   (4) — live, updated per step
dyn_feat                  (2) — live, updated per step
phase                     (1) — live
```

total	(135)
-------	-------

Dynamic features are normalized before concatenation: `days_worked / 223, consecutive_days / 5`.

Then two separate heads:

- **Actor:** `Linear(135, 64) → Tanh → Linear(64, 5) → 5 logits`. Illegal actions are masked to `-inf`, then softmax produces action probabilities.
- **Critic:** `Linear(135, 64) → Tanh → Linear(64, 1) → single scalar` estimating expected future reward from this state.

Why GNN over MLP?

An MLP would use learned `nn.Embedding(num_employees, dim)` — a fixed vector per employee ID. This works for one problem instance but breaks if you change the number of employees, swap team assignments, or use a different year.

The GNN computes embeddings from **features** (`days_worked, is_dual_team`) through message passing. It learns patterns like "employees with high remaining budget connected to days with large gaps should behave like X." A new employee with similar features gets a similar embedding without retraining — the GNN **generalizes across problem instances**.

Training Loop (PPO)

Proximal Policy Optimization (PPO) is an on-policy RL algorithm. Each training iteration collects a full trajectory using the current policy, updates the weights, then discards the trajectory — no replay buffer.

Trajectory Collection

The agent plays through an **entire episode** (Phase 1 + Phase 2) with gradients disabled. Every step records:

- **Inputs:** employee ID, day ID, coverage gaps, dynamic features, action mask, phase
- **Action:** sampled stochastically from the policy distribution
- **Log-probability** of the chosen action (needed for importance sampling during updates)
- **Reward** and **value estimate** from the critic
- **Graph feature snapshots:** employee and day node features at that moment, so the GNN can be accurately re-evaluated during PPO updates

A single trajectory is approximately $12 \times 365 = 4,380$ Phase 1 steps plus $\sim 2,000$ Phase 2 steps, totaling $\sim 6,000$ – $7,000$ steps per episode.

PPO Update

Once the trajectory is collected, advantages are computed using **Generalized Advantage Estimation (GAE)**. GAE measures how much better or worse each action was compared to the critic's prediction, using a weighted combination of short-term and long-term reward signals controlled by λ . With $\gamma = 1.0$ (no discounting), all future rewards matter equally — appropriate since early scheduling decisions affect the entire year. Advantages are normalized to stabilize training.

The trajectory is then reused for **K = 4 epochs**. In each epoch, the steps are shuffled into random **mini-batches of 512**. For each mini-batch:

1. **Restores graph features** from snapshots and re-runs the GNN to get fresh embeddings under current weights
2. Computes the **importance sampling ratio** between the new and old policy — correcting for the fact that actions were sampled under older weights
3. Applies the **clipped actor loss** — the core PPO mechanism that prevents the policy from changing too drastically per update by clamping the ratio to [0.8, 1.2]
4. Adds an **entropy bonus** that encourages exploration early in training (decays from 0.05 to 0.005 over episodes)
5. Computes **critic loss** (smooth L1) to improve value predictions that feed into future advantage estimates
6. Clips gradients to max norm 0.5 for stability

Mini-batches are used instead of full-trajectory updates for three reasons: lower memory usage, more frequent weight updates (~12 per epoch instead of 1), and decorrelation of consecutive steps (same-day steps are highly correlated; shuffling produces more independent gradients).

Key Hyperparameters

Parameter	Value	Purpose
Episodes	10,000	Total training episodes
Gamma (γ)	1.0	No discounting — entire year matters equally
Lambda (λ)	0.95	GAE bias-variance tradeoff
Clip epsilon	0.2	PPO clipping range [0.8, 1.2]
K epochs	4	Reuse each trajectory 4 times
Mini-batch size	512	Random mini-batches for gradient updates
Learning rate	3e-4	Adam optimizer (eps=1e-5)
Entropy coeff	0.05 → 0.005	Decays via $0.05 \times 0.999^{\text{episode}}$
Gradient clip	0.5	Max gradient norm
Value coeff	0.5	Weight of critic loss in total loss

Evaluation

Best-of-N stochastic sampling: run N episodes sampling from the policy distribution, keep the schedule with the best reward. This outperforms greedy (**argmax**) because the policy was trained with stochastic rollouts — sampling explores alternative assignment orders that produce globally better schedules in this highly constrained combinatorial problem.